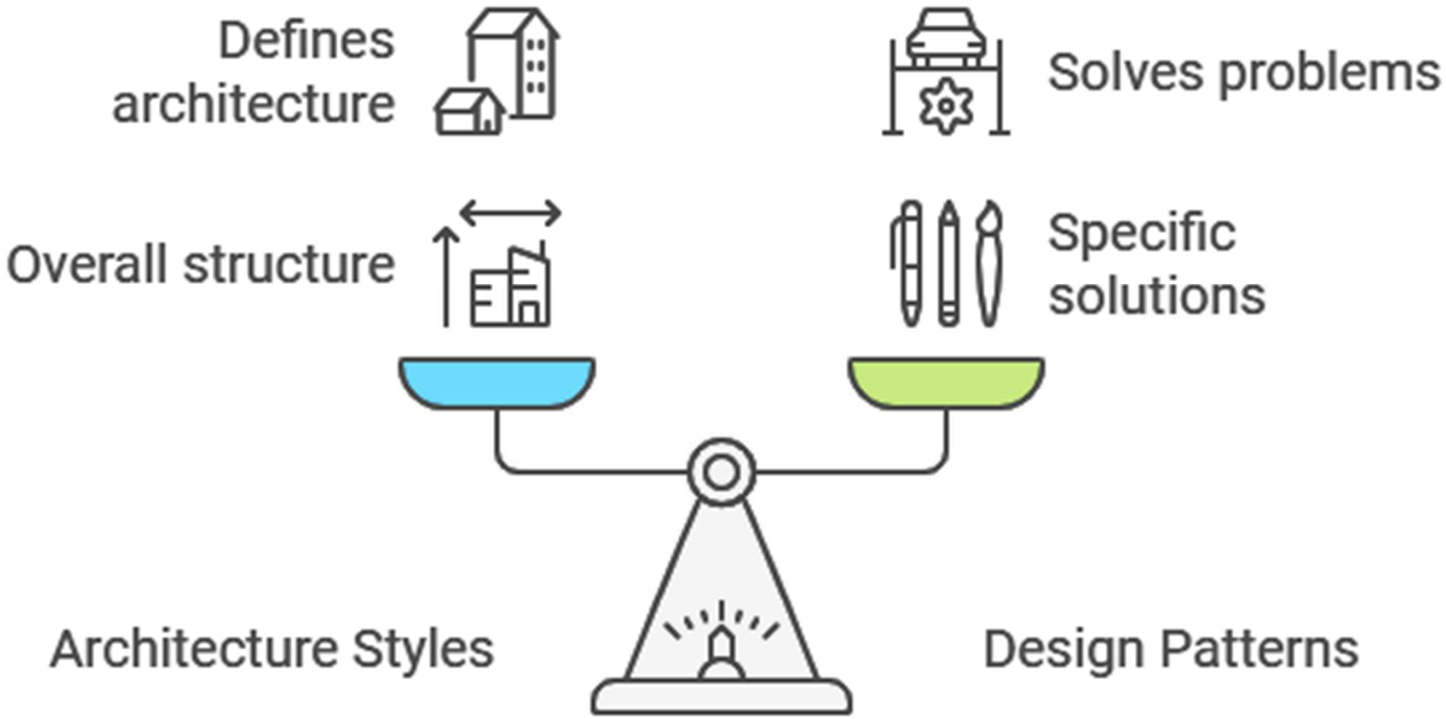


SOFWARE DESIGN & ARCHITECTURE

Chapter 7.1 – Structural Method in Architecture Design Pattern

Prepared By: Dr Muhammad Khatibsyarbini

ARCHITECTURAL STYLE vs PATTERN



Styles define structure; patterns provide solutions.

Structural design patterns focus on the composition and organization of classes and objects to create larger, complex structures. They ensure that parts of a system are structured in a way that optimizes flexibility, scalability, and reuse.

Structural Patterns

Patterns that define object composition and relationships.

Behavioral design patterns deal with **communication and interaction between objects**. They define how objects collaborate and interact to fulfill the system's functionality while adhering to principles like loose coupling and responsibility delegation.

Creational Patterns

Patterns focused on object creation mechanisms.



Behavioral Patterns

Patterns that manage object interaction and responsibility.

SUMMARY OF PATTERN

Creational pattern more to adding new object with repetitive procedure steps/methods.

Structural pattern is more like integration action by user like adding new component with different procedure/method.

Behavioral pattern, more to system behavior toward data, communication and interaction between objects normally indirectly trigger by user action.

ADAPTER PATTERN

•**Purpose**: Converts the interface of a class into another interface that clients expect, enabling classes with incompatible interfaces to work together.

•**Pros**:

- Enables reuse of existing code without modification.
- Provides flexibility by decoupling systems.

•**Cons**:

- Adds additional layers, potentially impacting performance.
- Can increase system complexity.

The step-by-step approach for designing the object structural adapter design pattern is presented as follows:

1. Identify the source and destination interfaces that need adapting in the new system (e.g., Target and Adaptee or Character and SpecialCharacter).
2. Add a new class (e.g., Adapter or AdaptedCharacter) in the design that realizes the Target interface and implements it in terms of the Adaptee's implementation. This requires a realization relationship between the Adapter and Target, and an association between the Adapter and the Adaptee.
3. In the new system, whenever objects that share the Target interface are expected, it can now be possible to use the Adapter objects created in step 2.

www.utm.my

ADAPTER PATTERN

Character is the Target class

```
class CharacterAdapter : public Character {
public:
  // Constructor.
  CharacterAdapter(BorrowedCharacter* pCharacter);

  // Adapt the attack method.
  void attack(void);

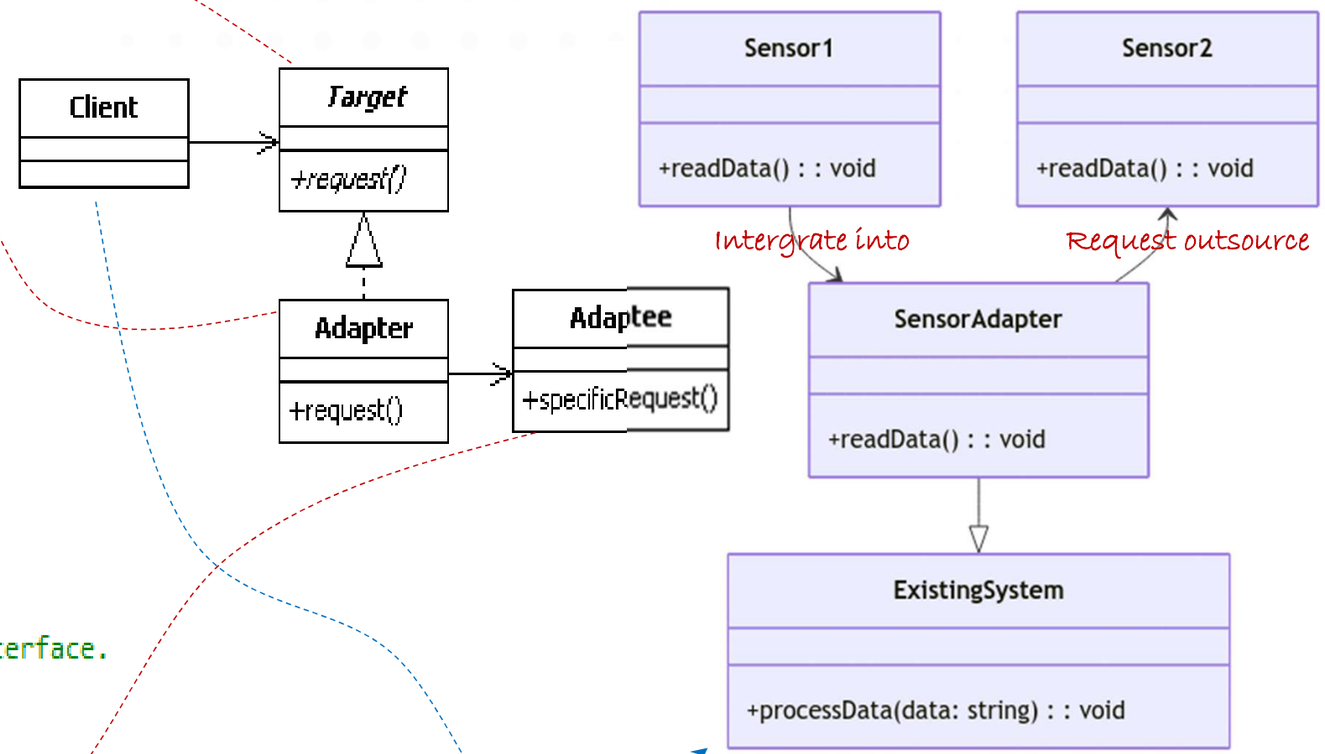
  // Adapt the defend method.
  void defend(void);

  // Adapt the move method.
  void move(void);

private:
  // BorrowedCharacter that needs adapting to the Character interface.
  BorrowedCharacter* _borrowedCharacter;
};
```

The CharacterAdapter

The class that we are adapting



www.utm.my

COMPOSITE PATTERN

•**Purpose**: Composes objects into tree-like structures to represent part-whole hierarchies, allowing clients to treat individual objects and composites uniformly.

•**Pros**:

- Simplifies client code by treating individual and composite objects the same.
- Scales well for hierarchical structures.

•**Cons**:

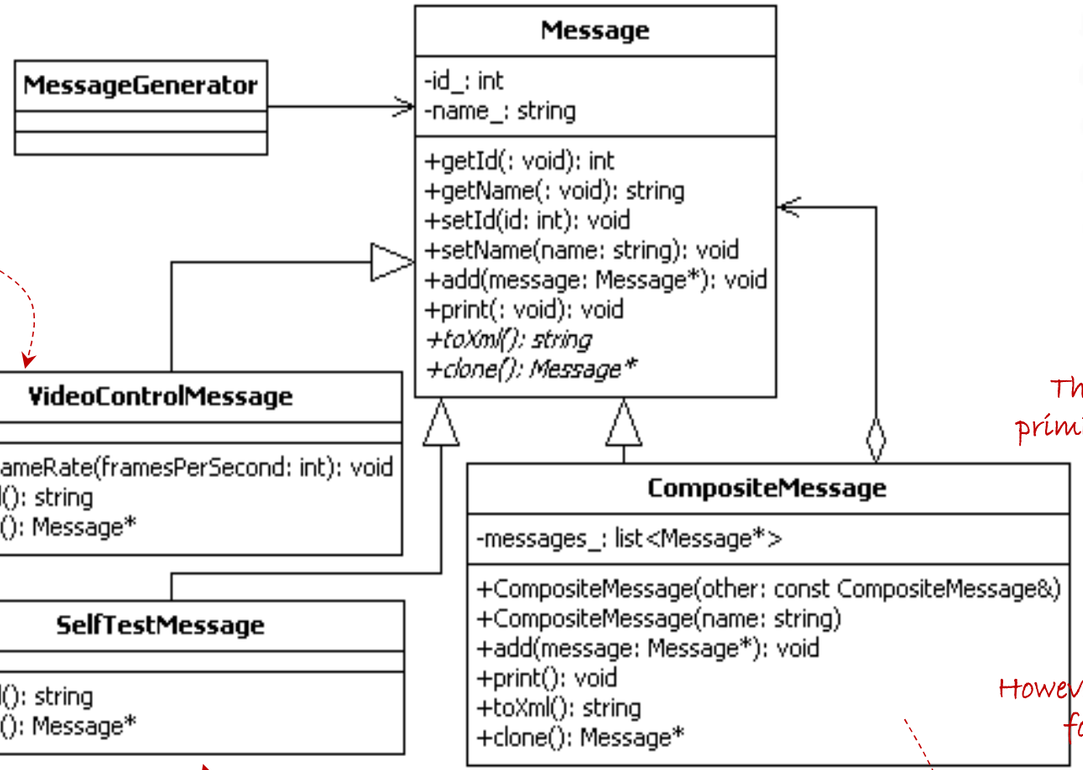
- Can make the system overly general.
- Difficult to restrict components to certain types.

- The Composite design pattern is an object structural pattern that allows designers to compose (large) tree-like designs structures by strategically structuring objects that share a whole-part relationship.

- According to the GoF, the intent of the Composite is to [1]
  - ✓ Compose objects into tree structures to represent whole-part hierarchies. Composite lets clients treat individual objects and composites of objects uniformly.

www.utm.my

COMPOSITE PATTERN



Primitive message for video collection control (Leaf)

Example of primitive messages

Primitive message for executing a system test to ensure everything is working OK (Leaf)

Composite message. This one is used to create composite messages, e.g., the mission plan message

These apply to both primitive and composite messages

However, consider these four methods

```
class Message {

public:

    // Constructor.
    Message(void);

    // Destructor.
    virtual ~Message(void);

    // Method to retrieve the message's id.
    int getId(void) const;

    // Method to retrieve the message's name.
    string getName(void) const;

    // Method to set the message's id.
    void setId(int id);

    // Method to set the message's name.
    void setName(string name);

    // Method to add messages to a composite message.
    virtual void add(Message* message);

    // Method to display messages to the console.
    virtual void print(void);

    // Method to transform the contents of this message to XML format.
    virtual string toXml(void) = 0;

    // Duplicate Messages using the prototype design pattern.
    virtual Message* clone(void) = 0;

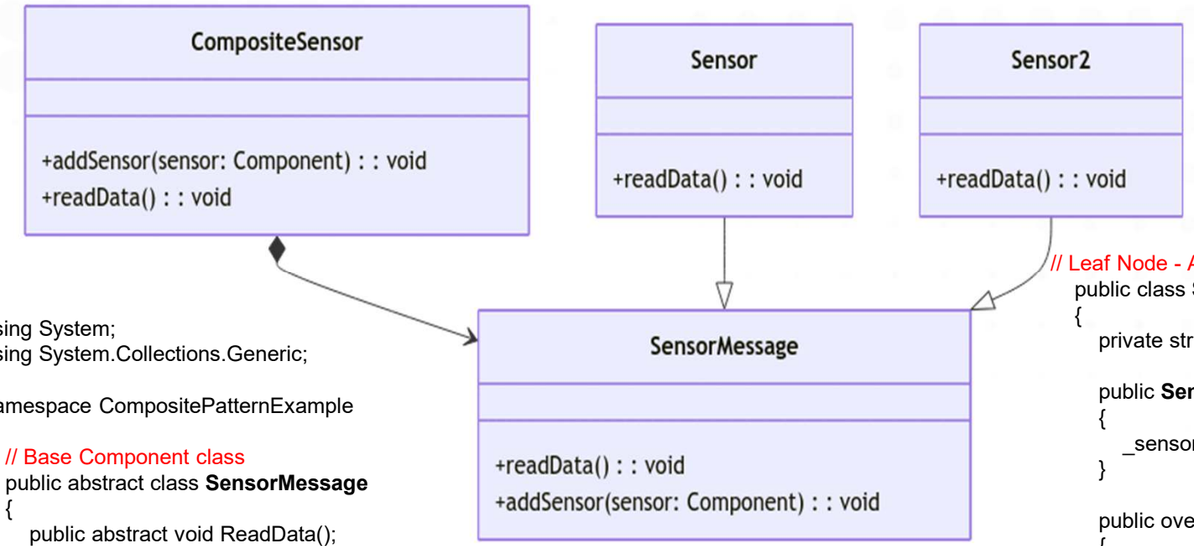
private:
    // The message's id.
    int _id;

    // The message's name.
    string _name;
};
```

Remember the Prototype design pattern?

www.utm.my

COMPOSITE PATTERN




```
using System;
using System.Collections.Generic;

namespace CompositePatternExample
{
    // Base Component class
    public abstract class SensorMessage
    {
        public abstract void ReadData();
        public virtual void AddSensor(SensorMessage sensor)
        {
            throw new NotImplementedException("AddSensor is not supported by this class.");
        }
    }

    // Leaf Node - Sensor
    public class Sensor : SensorMessage
    {
        private string _sensorName;

        public Sensor(string sensorName)
        {
            _sensorName = sensorName;
        }

        public override void ReadData()
        {
            Console.WriteLine($"{_sensorName} is reading data...");
        }
    }
```

```
// Leaf Node - Another type of Sensor
public class Sensor2 : SensorMessage
{
    private string _sensorName;

    public Sensor2(string sensorName)
    {
        _sensorName = sensorName;
    }

    public override void ReadData()
    {
        Console.WriteLine($"{_sensorName} is reading data...");
    }
}

// Composite class
public class CompositeSensor : SensorMessage
{
    private List<SensorMessage> _sensors = new List<SensorMessage>();

    public override void AddSensor(SensorMessage sensor)
    {
        _sensors.Add(sensor);
    }

    public override void ReadData()
    {
        Console.WriteLine("Composite Sensor reading data from all sensors:");
        foreach (var sensor in _sensors)
        {
            sensor.ReadData();
        }
    }
}
```

```
// Main Program
class Program
{
    static void Main(string[] args)
    {
        // Create individual sensors
        Sensor sensor1 = new Sensor("Temperature Sensor");
        Sensor sensor2 = new Sensor("Pressure Sensor");
        Sensor2 sensor3 = new Sensor2("Humidity Sensor");

        // Create a composite sensor
        CompositeSensor compositeSensor = new CompositeSensor();

        // Add individual sensors to the composite
        compositeSensor.AddSensor(sensor1);
        compositeSensor.AddSensor(sensor2);
        compositeSensor.AddSensor(sensor3);

        // Read data from all sensors
        compositeSensor.ReadData();
    }
}
```

www.utm.my

FACADE PATTERN

•**Purpose**: Provides a unified interface to a set of interfaces in a subsystem, simplifying the interaction with complex subsystems.
•**Pros**:
- Reduces client-side complexity.
- Promotes loose coupling between subsystems and clients.

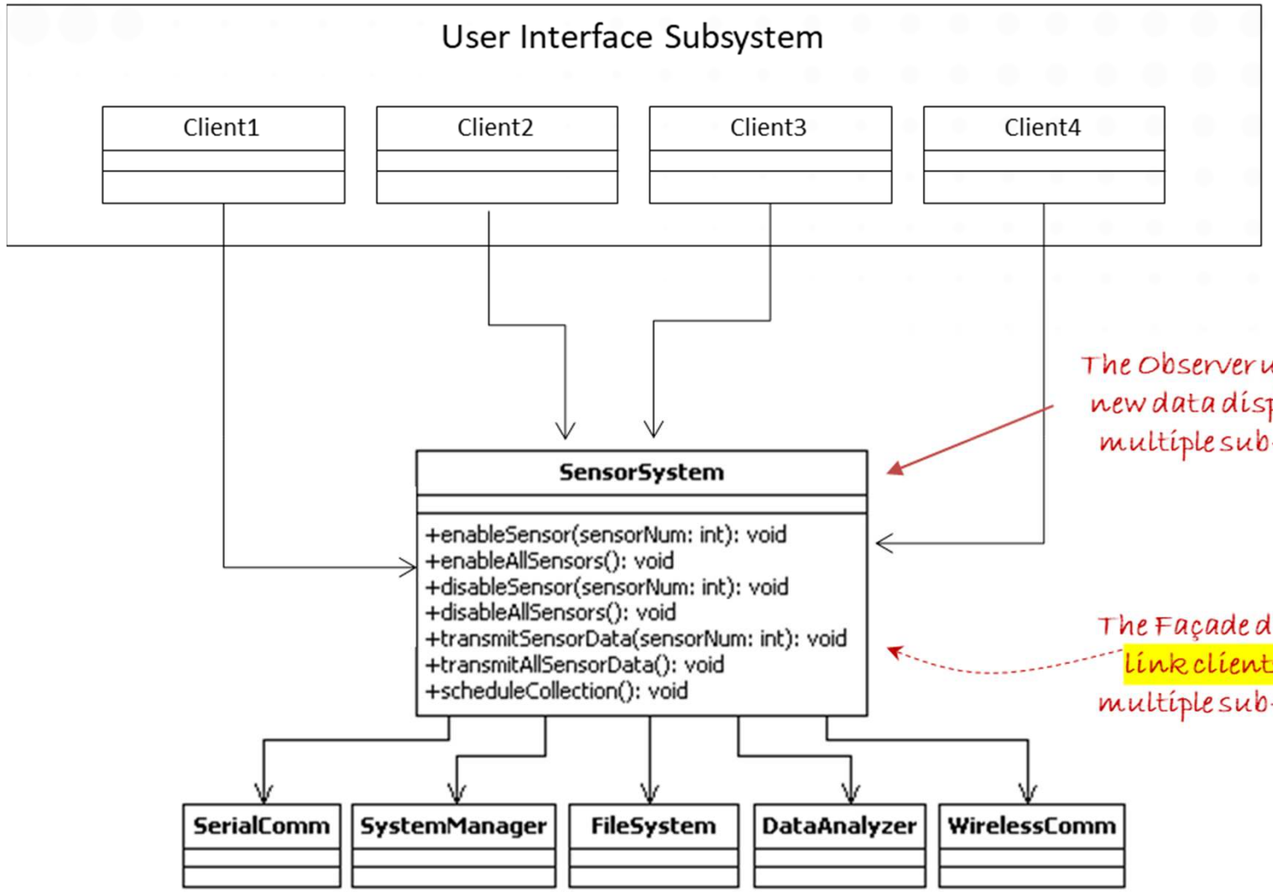
•**Cons**:
- May become a bottleneck if overloaded with too much responsibility.
- Limits flexibility if all subsystem features are required.

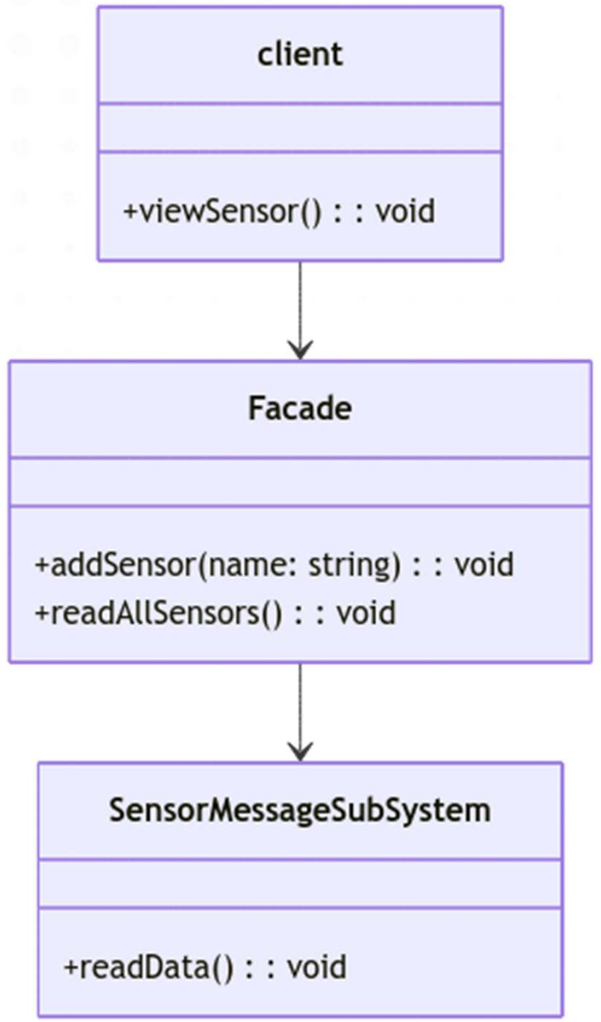
- The Facade design pattern is an object structural pattern that provides a simplified interface to complex subsystems.

- According to the GoF, the intent of the Facade is to [1]
  - ✓ Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.

www.utm.my

FACADE PATTERN





www.utm.my

FACADE PATTERN

```
using System;
using System.Collections.Generic;

namespace FacadePatternExample
{
    // Subsystem class
    public class SensorMessageSubSystem
    {
        private string _sensorName;

        public SensorMessageSubSystem(string sensorName)
        {
            _sensorName = sensorName;
        }

        public void ReadData()
        {
            Console.WriteLine($"{_sensorName} is reading data...");
        }
    }

    // Facade class
    public class Facade
    {
        private List<SensorMessageSubSystem> _sensors = new List<SensorMessageSubSystem>();

        public void AddSensor(string name)
        {
            _sensors.Add(new SensorMessageSubSystem(name));
        }

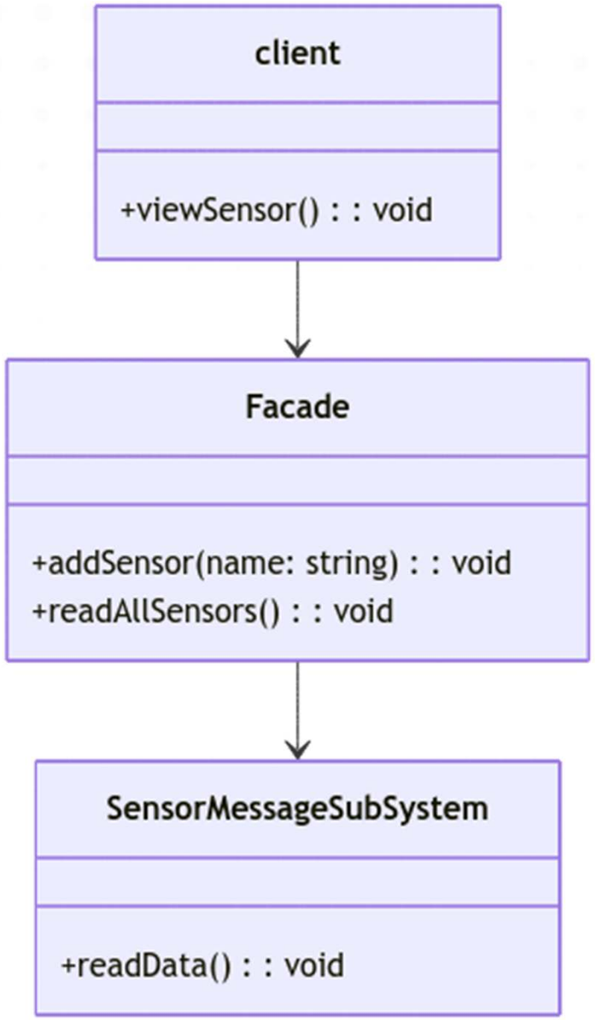
        public void ReadAllSensors()
        {
            Console.WriteLine("Reading data from all sensors through the facade:");
            foreach (var sensor in _sensors)
            {
                sensor.ReadData();
            }
        }
    }
```

```
    // Client class
    public class Client
    {
        public void ViewSensor()
        {
            Facade facade = new Facade();

            // Adding sensors
            facade.AddSensor("Temperature Sensor");
            facade.AddSensor("Pressure Sensor");
            facade.AddSensor("Humidity Sensor");

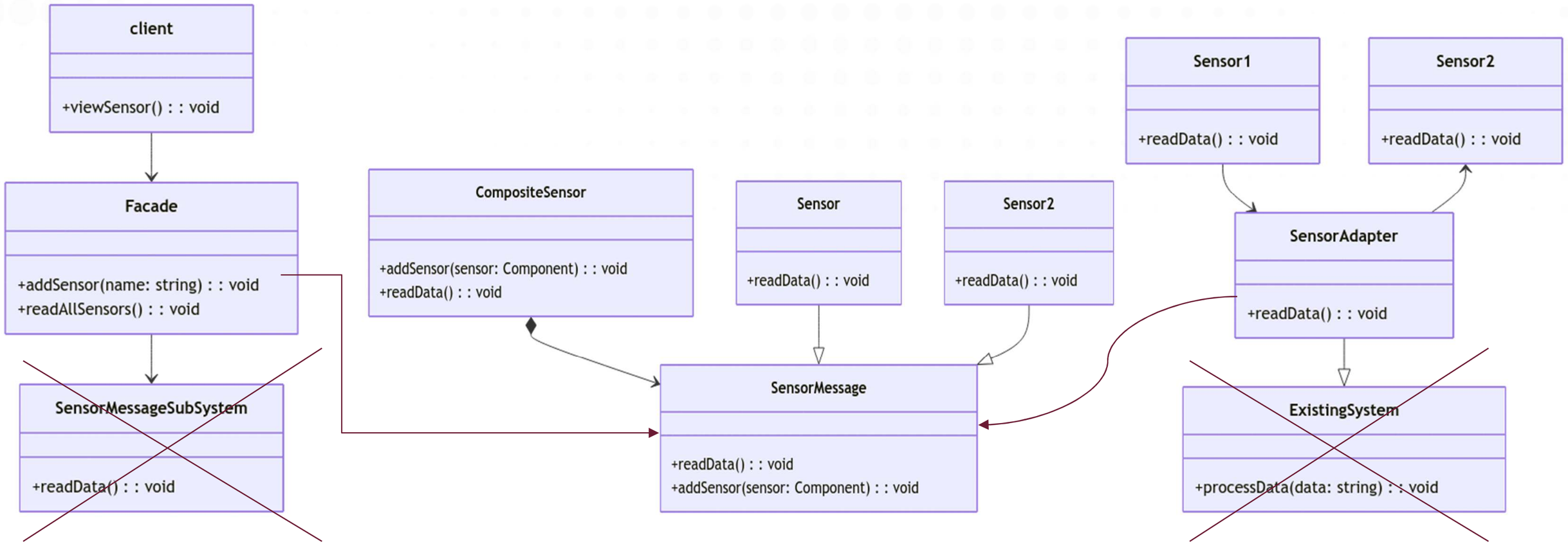
            // Reading data from all sensors
            facade.ReadAllSensors();
        }
    }

    // Main Program
    class Program
    {
        static void Main(string[] args)
        {
            // Client interacts with the system via the facade
            Client client = new Client();
            client.ViewSensor();
        }
    }
}
```



www.utm.my

COMBINED PATTERN



www.utm.my

THANK YOU

univteknologimalaysia utm.my utmofficial



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Innovating Solutions

Innovating Solutions

Menginovasi Penyelesaian

Menginovasi Penyelesaian